

Atomic Operators (Linux Kernel)

1. Introduction

- In a multiprocessor (SMP) system, multiple CPUs can access the same variable simultaneously.
- Standard C operations are **not atomic**.
- Concurrent access to shared variables may lead to **race conditions**.
- Linux provides **Atomic Operators** to safely manipulate shared variables without using locks.

2. The Problem: Read-Modify-Write Race Condition

Consider:

```
int i = 5;
```

```
i++;
```

The statement `i++` is not a single operation.

Internally it performs:

1. Read `i` from memory
2. Increment the value
3. Write the updated value back to memory

This sequence is called a **Read-Modify-Write (RMW)** operation.

3. Race Condition Example

Initial Value:

```
i = 5
```

CPU1

Read `i = 5`

Increment to 6

CPU2

Read `i = 5`

Increment to 6

Write Back

CPU1 writes 6

CPU2 writes 6

Final Value:

$i = 6$

Expected Value:

$i = 7$

One increment is lost.

This situation is called a **Race Condition**.

4. Why Hardware Arbitration is Not Enough

Modern systems use a **Memory Arbiter** to control access to memory.

The arbiter serializes:

Read Operations

Write Operations

However, it does **not** protect the complete:

Read $\rightarrow$ Modify $\rightarrow$ Write

sequence.

Therefore race conditions can still occur.

5. Hardware Support for Atomic Operations

Processors provide special atomic instructions.

These instructions guarantee that:

Read

Modify

Write

is executed as one indivisible operation.

No other CPU can access the memory location during the operation.

6. Atomic Operations on x86

x86 processors use the **LOCK** prefix.

Example:

lock incl variable

The LOCK prefix ensures:

Read
Modify
Write

occurs atomically.

Other CPUs are prevented from modifying the same memory location until the operation completes.

7. Atomic Operations on ARM

ARM processors use:

LL/SC

LL

Load Linked

SC

Store Conditional

Operation Flow:

1. Load value
2. Modify value
3. Attempt Store

If another CPU changes the value before the store:

Store fails

Operation retries

This prevents data corruption.

8. Linux Kernel Atomic API

Linux provides a synchronization mechanism called **Atomic Operators**.

These operators allow safe manipulation of shared variables without using spinlocks or mutexes.

Header File:

```
#include <linux/atomic.h>
```

9. Atomic Data Types

32-bit Atomic Variable

```
atomic_t counter;
```

64-bit Atomic Variable

```
atomic64_t counter64;
```

10. Initializing Atomic Variables

Method 1

```
atomic_t counter = ATOMIC_INIT(0);
```

Method 2

```
atomic_set(&counter, 0);
```

11. Common Atomic APIs

Read Value

```
atomic_read(&counter);
```

Example:

```
int val;
```

```
val = atomic_read(&counter);
```

Set Value

```
atomic_set(&counter, 10);
```

Increment

```
atomic_inc(&counter);
```

Equivalent to:

```
counter++;
```

but performed atomically.

Decrement

```
atomic_dec(&counter);
```

Equivalent to:

```
counter--;
```

but atomic.

Add

```
atomic_add(5, &counter);
```

Adds 5 atomically.

Subtract

```
atomic_sub(3, &counter);
```

Subtracts 3 atomically.

Increment and Return

```
atomic_inc_return(&counter);
```

Increments the value and returns the updated value.

Decrement and Return

```
atomic_dec_return(&counter);
```

Decrements the value and returns the updated value.

12. Example

```
#include <linux/atomic.h>
```

```
atomic_t counter = ATOMIC_INIT(0);
```

```
atomic_inc(&counter);
```

```
printk("Counter = %d\n",  
       atomic_read(&counter));
```

Output:

Counter = 1

13. Advantages

- Prevents race conditions.
- No explicit lock management.
- Faster than spinlocks for simple counters.

- Suitable for SMP systems.
- Low overhead.

14. Limitations

Atomic operations protect only **one variable at a time**.

Suitable for:

Counters
Statistics
Reference Counts
Flags

Not suitable for:

Multiple shared variables
Large critical sections
Complex synchronization

In such cases use:

Spinlocks
Mutexes
Semaphores

15. Atomic vs Spinlock

Atomic Operations	Spinlock
Protects single variable	Protects critical section
Very fast	Slower
No lock acquisition	Requires lock/unlock
Simple operations	Complex operations

16. Key Takeaways

- Standard C operations such as `i++` are not atomic.
- Concurrent execution can cause race conditions.
- Hardware provides atomic instructions (LOCK, LL/SC).
- Linux exposes these through the Atomic API.
- `atomic_t` is used for atomic integer operations.

- Atomic operations are ideal for counters, flags, and statistics.
- For complex shared data structures, use spinlocks or mutexes.

What is an Atomic Operation?

An atomic operation is an operation that executes completely as a single indivisible unit without interruption by other CPUs.

Why are Atomic Operators required?

To prevent race conditions when multiple CPUs access the same shared variable simultaneously.

What is atomic_t?

atomic_t is a Linux kernel data type used for performing atomic operations on 32-bit integers.

Which header file provides Atomic APIs?

```
#include <linux/atomic.h>
```

Name some Atomic APIs.

```
atomic_read()  
atomic_set()  
atomic_inc()  
atomic_dec()  
atomic_add()  
atomic_sub()  
atomic_inc_return()  
atomic_dec_return()
```

When should Atomic Operators be used?

When protecting a single shared variable such as counters, statistics, flags, or reference counts.

```

ability of vmlinux
make[1]: Leaving directory '/usr/src/linux-headers-6.8.0-124-generic'
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/atomic_operation$ ls
atomic_demo.c  atomic_demo.mod  atomic_demo.mod.o  Makefile  Module.symvers
atomic_demo.ko  atomic_demo.mod.c  atomic_demo.o  modules.order  README.md
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/atomic_operation$ sudo insmod atomic_demo.ko
[sudo] password for sethuraj:
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/atomic_operation$ sudo dmesg | tail -7
522667.986703] Initial Value = 5
522667.986704] After atomic_inc() = 6
522667.986704] After atomic_dec() = 5
522667.986704] After atomic_add(10) = 15
522667.986705] After atomic_sub(3) = 12
522667.986705] After atomic_set(100) = 100
522667.986706] =====
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/atomic_operation$ sudo rmmod atomic_demo
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/atomic_operation$ sudo dmesg | tail -9
sudo: dmesg: command not found
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/atomic_operation$ sudo dmesg | tail -9
===== Atomic Operations Demo =====
522667.986703] Initial Value = 5
522667.986704] After atomic_inc() = 6
522667.986704] After atomic_dec() = 5
522667.986704] After atomic_add(10) = 15
522667.986705] After atomic_sub(3) = 12
522667.986705] After atomic_set(100) = 100
522667.986706] =====
522714.457004] Atomic Demo Module Removed
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/atomic_operation$ _

```

Atomic Bit Operations (Bit-Level Atomic Operations)

After learning `atomic_t`, Linux provides another synchronization mechanism called **Atomic Bit Operations**.

These are used when you want to atomically manipulate **individual bits** instead of entire integers.

```
#include <linux/bitops.h>
```

Atomic Integer	Atomic Bit
<code>atomic_t</code>	<code>bitops</code>
Operates on entire integer	Operates on individual bits
<code>atomic_inc()</code>	<code>set_bit()</code>
<code>atomic_dec()</code>	<code>clear_bit()</code>
Used for counters	Used for flags

What are Atomic Bit Operations?

Atomic bit operations are kernel synchronization primitives that atomically manipulate individual bits of a memory location without race conditions.

set_bit() → Set bit
clear_bit() → Clear bit
change_bit() → Toggle bit
test_bit() → Check bit
test_and_set_bit() → Check + Set
test_and_clear_bit() → Check + Clear
test_and_change_bit() → Check + Toggle

```
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/atomic_operation/atomic_bit$  
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/atomic_operation/atomic_bit$ ls  
atomic_bitops.c  atomic_bitops.mod  atomic_bitops.mod.o  Makefile  Module.symvers  
atomic_bitops.ko  atomic_bitops.mod.c  atomic_bitops.o  modules.order  
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/atomic_operation/atomic_bit$ sudo insmod atomic_b  
o  
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/atomic_operation/atomic_bit$ sudo dmesg | tail -1  
[523472.445278] =====  
[523499.418755] Atomic Bit Operations Module Removed  
[523515.764859]  
        ===== Atomic Bit Operations Demo =====  
[523515.764863] Initial flags = 0  
[523515.764864] After set_bit(0)      = 1  
[523515.764864] After set_bit(2)      = 5  
[523515.764865] Bit 0 is SET  
[523515.764865] After clear_bit(0)    = 4  
[523515.764865] After change_bit(1)    = 6  
[523515.764865] Old value of bit3 = 0  
[523515.764866] After test_and_set_bit(3) = e  
[523515.764866] Old value of bit2 = 1  
[523515.764866] After test_and_clear_bit(2) = a  
[523515.764867] =====  
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/atomic_operation/atomic_bit$
```